

Technical Guide: Atari SIO2PC USB Interface

Subtitle: Future-Proofing Atari 8-Bit Connectivity with AspeQt-2k26

Author: 13leader.net

Date: March 2026

Project Repo: <https://github.com/pjones1063/AspeQt-2k26>

Building an Atari SIO2PC cable is a straightforward and cost-effective way to connect your vintage Atari hardware to a modern PC running Windows, Linux, or macOS. By using a USB-to-TTL chipset, you can utilize the AspeQt-2k26 emulator to serve disk images and manage files directly on real Atari hardware.

Our BBS and project updates can be found at: <https://13leader.net>

1. Parts & Requirements

- **USB-to-TTL Adapter:** Common chipsets include the **PL2303TA** or the **FT232RL**.
- **Atari SIO Connector:** Required for the cable interface. These can be sourced from eBay or specialty vendors like [Best Electronics](#).
- **AspeQt-2k26 Software:** Compatible with RespeQt and legacy AspeQt emulators.

2. Build Option A: PL2303TA Chipset

This version is widely compatible with **Windows, Linux, and macOS** for standard disk emulation.

Wiring Specifications

Atari SIO Pin	Signal Description	PL2303TA Wire Color
Pin 3	Data In	TXD (Green)
Pin 4	Ground	GND (Black)
Pin 5	Data Out	RXD (White)

Software Setting: In AspeQt, set **Handshake** to "None".

3. Build Option B: FTDI FT232RL Chipset – 850 R:

The FTDI version is highly recommended for advanced users as it is also compatible with **APE Atarimax** software and supports software 850 R: emulation within AspeQt-2K26.

Wiring Specifications

Atari SIO Pin	Signal Description	FT232RL Pin
Pin 3	Data In	TXD
Pin 4	Ground	GND
Pin 5	Data Out	RXD
Pin 7	Command Signal	CTS

Software Setting: In AspeQt, set **Handshake** to "CTS".

Software Setting: In AspeQt, set Handshake to "CTS".

R: Device Emulation & CTS Logic (New for AspeQt-2k26): By connecting the CTS pin (Pin 7) as shown above, your FT232 cable fully supports AspeQt-2k26's native 850 R: Device emulation (Virtual Modem) for BBS connectivity.

Important Note on CTS Inversion: Legacy Atari software was built around physical RS-232 adapters that physically inverted logic signals. Most modern, generic FT232 breakout boards use standard TTL logic (non-inverted). If your R: Device emulation instantly drops or exits stream mode, the software is expecting an inverted signal. To fix this, simply toggle the "Invert CTS Logic" setting in the AspeQt-2k26 options, or use the free FT_PROG utility to flash the hardware EEPROM on your FTDI chip to invert the CTS pin permanently.

4. Software Connectivity (All Platforms)

Driver Installation: Ensure your OS recognizes the USB chipset (FTDI or Prolific).

Launch AspeQt-2k26: * Serial Port: Select the COM port (Windows) or /dev/ttyUSBx (Linux/macOS) associated with your cable.

Baud Rate: Standard start is 19,200. Once verified, AspeQt-2k26 supports high-speed SIO routines.

Integrated TNFS: Use the built-in browser to pull files directly from the web to your Atari.

5. Troubleshooting

Handshake Mismatch: If the cable fails to "see" the Atari, double-check your Handshake setting (None for PL2303, CTS for FTDI).

Wiring Check: Ensure TXD and RXD are correctly swapped (Atari Data In connects to USB TXD).

OS Compatibility: The FT232RL chipset generally has better driver support across modern macOS and Linux versions.

OS Performance & Success Rates (R: Device Emulation)

When using the virtual modem, entering SIO stream mode is 100% reliable across all hardware and platforms. However, because modern USB stacks buffer serial data, exiting SIO stream mode (such as dropping out of a terminal program like Ice-T back to DOS, or updating baud rates) is highly susceptible to latency timeouts.

Depending on your host operating system, expect the following stability levels when **exiting stream mode**:

Raspberry Pi (Direct UART): 100% Success (The Gold Standard - Zero USB latency).

Debian / Linux (FT232): ~95% Success (Very stable, minor occasional timeouts).

macOS (FT232): ~75% Success (Occasional timeouts).

Windows (FT232): ~45% Success (Frequent timeouts due to OS-level USB polling delays).

Windows Troubleshooting Tip: The official Windows FTDI driver defaults to a 16ms Latency Timer, which causes the frequent timeouts mentioned above. To dramatically

improve stability on Windows: Open Device Manager -> Ports (COM & LPT) -> Right-click your USB Serial Port -> Properties -> Port Settings tab -> Advanced -> Change "Latency Timer (msec)" from 16 to 1.